

Projeto Final de Disciplina — Garantia de Qualidade de Software

Título

Projeto Integrado de Garantia e Avaliação da Qualidade de Software

Disciplina: Garantia de Qualidade de Software

Professor: Christian Azonyetin

Modalidade: Trabalho em grupo — máximo de 3 alunos

1. Objetivo Geral

Desenvolver e aplicar um **processo completo de garantia da qualidade de software**, contemplando planejamento da qualidade, definição de critérios de aceitação, validação de requisitos, revisão técnica, análise estática, identificação e classificação de defeitos, testes caixa-preta e caixa-branca, testes em diferentes níveis, automação de testes, métricas de qualidade, análise dos resultados e melhoria contínua.

O projeto deverá demonstrar que os estudantes são capazes não apenas de **testar um software**, mas de **avaliar sistematicamente sua qualidade e produzir evidências técnicas para apoiar decisões de engenharia**.

2. Descrição do Projeto

Cada grupo deverá selecionar ou desenvolver uma **aplicação de pequeno/médio porte** que será utilizada como **Sistema sob Teste — SUT (System Under Test)**.

Exemplos:

- sistema de biblioteca;
- sistema acadêmico;
- sistema de vendas;
- sistema de agendamento;
- sistema de gerenciamento de estoque;
- sistema de atendimento;
- sistema de controle financeiro;
- sistema de gestão de tarefas;
- sistema de gerenciamento de serviços.

Importante

O objetivo principal **não é o desenvolvimento da aplicação**. A aplicação é o **objeto sobre o qual será aplicado o processo de garantia de qualidade**. O trabalho deverá responder, tecnicamente:

"Como podemos demonstrar, por meio de evidências, que este software atende aos requisitos e apresenta um nível aceitável de qualidade?"

3. Estrutura geral do projeto

O projeto deverá seguir aproximadamente este ciclo:

REQUISITOS



PLANEJAMENTO DA QUALIDADE



CRITÉRIOS DE ACEITAÇÃO



PLANO DE TESTES



REVISÃO TÉCNICA



ANÁLISE ESTÁTICA



PROJETO DOS CASOS DE TESTE



TESTES CAIXA-PRETA



TESTES CAIXA-BRANCA



TESTES UNITÁRIOS



TESTES DE INTEGRAÇÃO



TESTES DE SISTEMA



TESTES DE ACEITAÇÃO



TESTES DE REGRESSÃO



AUTOMAÇÃO



COLETA DE MÉTRICAS



ANÁLISE DOS RESULTADOS



IDENTIFICAÇÃO DE DEFEITOS



CORREÇÕES



RETESTES



RELATÓRIO FINAL DE QUALIDADE

4. Requisitos obrigatórios

4.1. Definição do sistema

O grupo deverá apresentar:

- descrição do sistema;
- problema que o sistema resolve;
- público-alvo;
- funcionalidades;
- requisitos funcionais;
- requisitos não funcionais;
- restrições;
- riscos associados ao sistema.

Deverão existir pelo menos:

- **10 requisitos funcionais;**
- **5 requisitos não funcionais.**

4.2. Planejamento da qualidade

O grupo deverá produzir um **Plano de Qualidade de Software** contendo, no mínimo:

- objetivos de qualidade;
- atributos de qualidade avaliados;
- critérios de aceitação;
- estratégia de testes;
- níveis de teste;
- ferramentas utilizadas;
- responsabilidades;
- riscos;
- critérios de entrada;
- critérios de saída;
- critérios para aprovação/reprovação.

Os atributos deverão considerar, conforme aplicável:

- confiabilidade;
- usabilidade;
- eficiência;
- manutenibilidade;
- portabilidade;
- segurança.

4.3. Critérios de aceitação

Cada requisito importante deverá possuir critérios objetivos.

Por exemplo:

RF-05 — Cadastro de usuário_Critérios de aceitação:

CA-01 — O sistema deve aceitar nome válido.

CA-02 — O sistema deve rejeitar nome vazio.

CA-03 — O sistema deve impedir CPF duplicado.

CA-04 — O sistema deve informar claramente o erro.

CA-05 — O cadastro válido deve ser persistido.

Isso permitirá posteriormente relacionar: **Requisito** → **Caso de teste** → **Resultado** → **Defeito**

5. Validação dos requisitos

Antes de testar o código, os estudantes deverão testar **os próprios requisitos**.

Deverão identificar problemas como:

- ambiguidades;
- requisitos incompletos;
- requisitos contraditórios;
- requisitos impossíveis de verificar;
- ausência de critérios de aceitação;
- termos subjetivos.

Exemplo: “O sistema deve ser rápido.”

Isso é um requisito ruim porque “rápido” não possui uma definição mensurável.

O grupo deverá transformá-lo em algo verificável: “95% das consultas deverão responder em até 2 segundos sob a carga definida no plano de teste.”

6. Revisão técnica e inspeção

O grupo deverá realizar uma **revisão técnica formal**.

Deverão ser analisados, no mínimo:

- requisitos;
- arquitetura;
- código;
- casos de teste;
- documentação.

Cada problema encontrado deverá ser registrado. Registro mínimo de defeitos:

ID	Artefato	Descrição	Severidade	Prioridade	Status
D001	Código	Validação ausente	Alta	Alta	Corrigido
D002	Requisito	Ambíguo	Média	Alta	Corrigido
D003	Interface	Mensagem inadequada	Baixa	Média	Aberto

7. Análise estática

Essa parte é **obrigatória**.

Os grupos deverão utilizar pelo menos uma ferramenta de análise estática para identificar problemas no código sem executar o programa.

Exemplos:

- SonarQube/SonarCloud;
- Pylint;
- Ruff;
- ESLint;
- PMD;
- Checkstyle;
- ferramentas equivalentes.

Deverão identificar, quando aplicável:

- código duplicado;
- complexidade;
- problemas de estilo;
- possíveis bugs;
- código morto;
- vulnerabilidades;
- problemas de manutenção.

E deverão comparar:

ANTES DA CORREÇÃO

↓

ANÁLISE ESTÁTICA

↓

CORREÇÕES

↓

NOVA ANÁLISE

↓

DEPOIS DA CORREÇÃO

8. Testes caixa-preta

O grupo deverá utilizar técnicas de projeto de testes **sem considerar a implementação interna do código**.

Deverão aplicar pelo menos **3 técnicas** dentre:

- particionamento de equivalência;
- análise de valor-limite;
- tabela de decisão;
- teste baseado em requisitos;
- teste baseado em risco;
- testes de casos inválidos;

- testes de exceção.

Exemplo

Para idade: Regra: idade permitida = 18 até 60 anos

Os casos não devem ser simplesmente: 18; 30; 40; 50; 60

Devem explorar principalmente fronteiras: 17; 18; 19; 59; 60; 61

Isso demonstra efetivamente domínio da técnica.

9. Testes caixa-branca

O grupo deverá analisar a **estrutura interna do código**.

Deverá demonstrar, por exemplo:

- caminhos de execução;
- decisões;
- condições;
- cobertura de instruções;
- cobertura de branches/decisões.

Deverão produzir evidências de cobertura.

Exemplo:

Cobertura antes:

Statements: 72%

Branches: 58%

↓

Novos casos de teste

↓

Cobertura depois:

Statements: 94%

Branches: 89%

Os valores acima são apenas ilustrativos. Os grupos deverão apresentar os valores reais obtidos.

10. Estratégia baseada em risco

Esta é uma parte que considero **muito importante** para tornar o projeto mais profissional. Os grupos deverão classificar funcionalidades de acordo com o risco.

Funcionalidade	Probabilidade	Impacto	Risco
Login	Alta	Alta	Crítico
Pagamento	Média	Alta	Alto
Consulta	Média	Média	Médio
Alteração de tema	Baixa	Baixa	Baixo

A intensidade dos testes deverá ser proporcional ao risco. Assim, os alunos deverão justificar: **Por que determinada funcionalidade recebeu mais testes que outra?**

11. Casos de teste

Deverão ser elaborados pelo menos **30 casos de teste**, distribuídos entre diferentes categorias. Cada caso deverá conter:

ID; Requisito relacionado; Objetivo; Pré-condições; Dados de entrada; Passos; Resultado esperado; Resultado obtido; Status; Evidência.

Exemplo:

ID	Requisito	Entrada	Resultado esperado	Resultado
CT001	RF01	Dados válidos	Cadastro realizado	Passou
CT002	RF01	Nome vazio	Mensagem de erro	Passou
CT003	RF01	CPF duplicado	Cadastro rejeitado	Falhou

12. Níveis de teste

O projeto deverá contemplar:

- 1. Teste unitário** (Testar componentes/funções individualmente).
- 2. Teste de integração** (Verificar a interação entre componentes).
- 3. Teste de sistema** (Avaliar o sistema completo).
- 4. Teste de aceitação** (Verificar se os requisitos e critérios de aceitação foram atendidos.)
- 5. Teste de regressão** (Depois de realizar correções, verificar se funcionalidades anteriormente aprovadas continuam funcionando).

13. Testes automatizados

A automação deverá ser **obrigatória**. Não basta executar testes manualmente e colocar screenshots no relatório. O grupo deverá desenvolver uma suíte de testes automatizados.

Por exemplo, utilizando:

Python



pytest



coverage



testes automatizados

Ou outra stack válido. A suíte deverá conter testes:

- positivos;
- negativos;
- casos-limite;
- regressão;
- diferentes níveis de teste, conforme aplicável.

14. Automação de testes

O grupo deverá demonstrar a execução automática:

Executar testes



Sistema executa dezenas de casos



Resultados



PASS / FAIL



Relatório

Idealmente, a execução deverá ser integrada a um pipeline de CI. Por exemplo:

Git



Commit



Pipeline



Análise estática

↓

Testes automatizados

↓

Cobertura

↓

Resultado

15. Defeitos

Os grupos deverão obrigatoriamente encontrar e registrar defeitos. Para cada defeito:

ID; Título; Descrição; Passos para reprodução; Resultado esperado; Resultado obtido; Severidade; Prioridade; Evidência; Responsável; Status; Correção; Reteste.

E deverão demonstrar o ciclo:

DEFEITO

↓

REGISTRO

↓

ANÁLISE

↓

CORREÇÃO

↓

RETESTE

↓

REGRESSÃO

↓

ENCERRAMENTO

16. Métricas de qualidade

O projeto deverá obrigatoriamente coletar métricas.

Por exemplo:

Métricas de testes

- quantidade de casos de teste;
- testes executados;

- testes aprovados;
- testes reprovados;
- taxa de aprovação;
- taxa de falha.

Métricas de defeitos

- defeitos encontrados;
- defeitos por severidade;
- defeitos corrigidos;
- defeitos abertos;
- densidade de defeitos.

Métricas de código

- cobertura de testes;
- complexidade;
- duplicação;
- problemas encontrados pela análise estática.

Exemplo:

Taxa de aprovação =

$$\frac{\text{testes aprovados}}{\text{testes executados}} \times 100$$

Os grupos deverão **interpretar** as métricas. Não basta apresentar: “A cobertura foi 87%.”

Eles precisam responder: “O que significa 87% para este projeto? É suficiente? Quais partes continuam sem cobertura? Qual é o risco associado?”

17. Avaliação dos atributos de qualidade

Cada grupo deverá escolher métricas para avaliar os atributos mais relevantes do sistema.

Por exemplo:

Atributo	Indicador
Confiabilidade	Taxa de falhas
Usabilidade	Taxa de conclusão das tarefas
Eficiência	Tempo de resposta
Manutenibilidade	Complexidade/código duplicado
Segurança	Vulnerabilidades encontradas
Portabilidade	Execução em diferentes ambientes

Isso faz a ponte entre: “**testar funcionalidades**” e “**avaliar qualidade de software**”.

18. Segurança, privacidade e ética

O grupo deverá analisar aspectos éticos relacionados ao sistema.

Deverão discutir:

- privacidade;
- proteção de dados;
- segurança;
- coleta de dados;
- armazenamento;
- exposição de informações;
- responsabilidade profissional;
- possíveis impactos sociais;
- riscos de discriminação ou decisões injustas.

Também deverão responder: **Quais consequências poderiam ocorrer caso este software apresentasse uma falha grave?**

19. Melhoria contínua

O projeto não deverá terminar quando os testes forem executados. Os grupos deverão realizar pelo menos um ciclo de melhoria:

Versão inicial

↓

Testes

↓

Defeitos

↓

Métricas

↓

Análise

↓

Correções

↓

Nova execução

↓

Comparação

E apresentar: Antes × Depois:

Indicador	Antes	Depois
Cobertura	X%	Y%
Falhas	X	Y
Defeitos críticos	X	Y
Complexidade	X	Y
Problemas estáticos	X	Y

Assim conseguimos verificar se **as ações de melhoria realmente produziram resultados**.

20. Requisitos técnicos mínimos

Eu estabeleceria os seguintes requisitos:

Obrigatórios

- sistema executável;
- código-fonte;
- controle de versão;
- requisitos documentados;
- plano de qualidade;
- plano de testes;
- casos de teste;
- testes caixa-preta;
- testes caixa-branca;
- testes baseados em risco;
- testes unitários;
- testes de integração;
- testes de sistema;
- testes de aceitação;
- testes de regressão;
- testes automatizados;
- análise estática;
- registro de defeitos;
- métricas;
- análise dos resultados;
- ciclo de melhoria;
- documentação.

21. Stack tecnológica recomendada

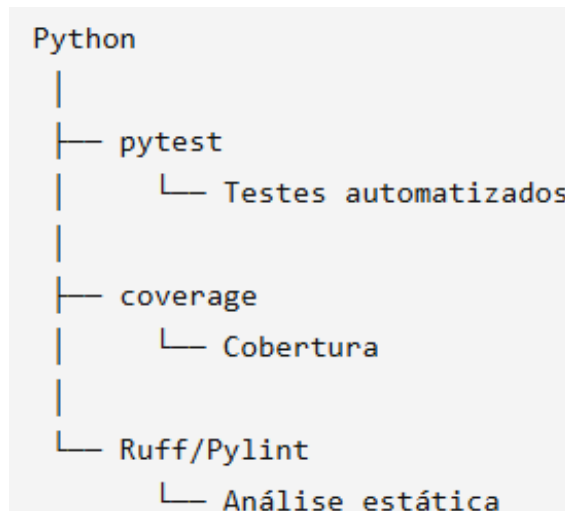
Para manter o foco na disciplina, eu **não obrigaria uma linguagem específica de programação**.

Porém, para padronizar e facilitar a avaliação, minha recomendação seria:

Stack padrão

Python + pytest + coverage + ferramenta de análise estática + Git

Por exemplo:



Para testes de interface web, poderia ser acrescentado: **Playwright** ou ferramenta equivalente.

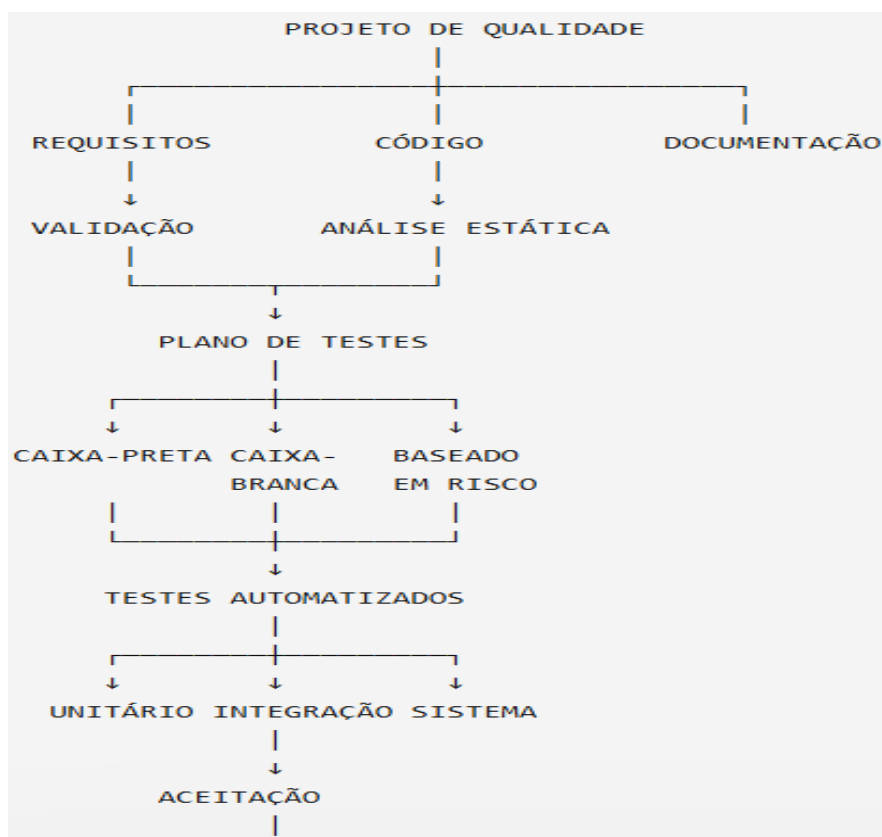
Por que Python?

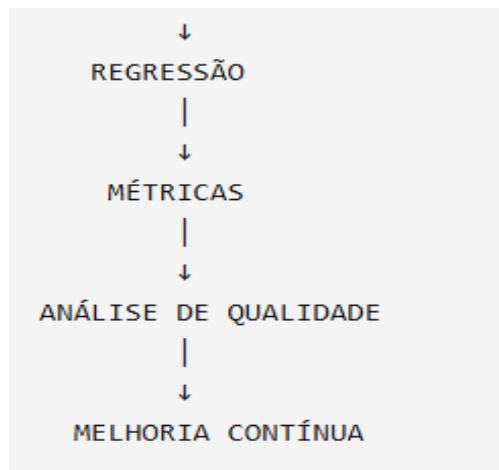
Porque permite que os alunos concentrem o esforço em:

- engenharia de testes;
- projeto de casos;
- automação;
- análise;
- métricas;
- qualidade;

em vez de perder grande parte do projeto aprendendo uma linguagem excessivamente complexa.

22. Arquitetura do projeto





23. Critérios de avaliação

Critério	Pontos
Requisitos e critérios de aceitação	8
Plano de qualidade	8
Planejamento e estratégia de testes	8
Testes caixa-preta	10
Testes caixa-branca	10
Testes baseados em risco	7
Testes unitários, integração, sistema e aceitação	10
Testes automatizados	12
Análise estática e revisão técnica	7
Gestão de defeitos	5
Métricas e indicadores	7
Melhoria contínua e comparação antes/depois	4
Ética, segurança e responsabilidade profissional	2
Documentação e apresentação	2
Total	100

24. Entrega final

Cada grupo deverá entregar (**Até 30 de novembro**):

1. Código-fonte

Repositório contendo: /src

/tests

/docs

/reports

/config

README.md

A estrutura poderá variar conforme a tecnologia.

2. Plano de qualidade

3. Plano de testes

4. Catálogo de casos de teste

5. Relatório de defeitos

6. Relatório de análise estática

7. Relatórios de testes automatizados

8. Relatório de métricas

9. Relatório técnico final

10. Apresentação e demonstração prática

25. Regra

Todos os resultados apresentados deverão ser obtidos a partir da execução real do software e das ferramentas utilizadas. Resultados, métricas, capturas de tela, relatórios ou evidências que não possam ser reproduzidos durante a apresentação não serão considerados válidos.

E, durante a apresentação, seria solicitado:

- executar um teste;
- criar um novo caso de teste;
- alterar um valor;
- provocar uma falha;
- executar a suíte automatizada;
- mostrar a cobertura;
- localizar um defeito;
- explicar uma métrica;
- justificar um caso de teste;
- demonstrar um teste caixa-preta;
- demonstrar um teste caixa-branca;
- mostrar a análise estática;
- explicar por que determinada funcionalidade possui maior risco.

Isso reduz muito a possibilidade de o grupo simplesmente **copiar um projeto pronto ou apresentar números inventados.**